

Project Interim Report
On
Gram-Sewa —
Hyperlocal Civic Issue Tracker

RU-25-10436
Frontend Frameworks (React)

SESSION: 2025-26



Submitted By—
D Rupesh Kumar
Reg No. 02033

**Master of Computer Application with Full Stack in
Association With Google**

Under the Guidance of
Mr. Ashutosh Banergee
SWE EXPERT

SCHOOL OF INFORMATION TECHNOLOGY
RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Abstract

Gram-Sewa ("Village Service") is a hyperlocal civic issue tracking platform designed to empower rural citizens across India. The platform enables villagers to report infrastructure problems — such as damaged roads, water supply failures, electricity outages, and sanitation issues — through a simple, mobile-friendly web interface with image upload support. Built using a modern full-stack architecture with React and Tailwind CSS on the frontend, and Express.js with MongoDB on the backend, Gram-Sewa bridges the communication gap between rural communities and local governing bodies like Gram Panchayats and Block Development Offices. Each complaint progresses through a transparent status pipeline — Pending, In Review, In Progress, and Resolved — allowing citizens to track resolution in real time. Authorities receive a centralized dashboard with filtering, priority assignment, and analytics capabilities. The platform addresses a critical need: giving India's rural population a structured, digital voice for civic grievances while introducing accountability into local governance.

Introduction

In rural India, civic infrastructure complaints — broken roads, contaminated water supply, erratic electricity, and overflowing drains — are typically communicated verbally to local ward members or Gram Panchayat officials. These complaints lack any documentation trail, making it impossible for citizens to track whether their grievances have been acknowledged or acted upon. The absence of a structured reporting mechanism leads to repeated complaints, delayed resolutions, and citizen disengagement from local governance processes. Gram-Sewa addresses this problem by providing a simple, mobile-friendly web interface where citizens can file structured complaints with photographic proof. In this Phase 1 release, the entire application runs on the client side — React handles the UI, Tailwind CSS provides responsive styling, and localStorage persists all data within the browser. A CSV import/export module ensures that data is not permanently lost when switching browsers or clearing cache.

Key Objectives:

- Enable citizens to report infrastructure issues with title, description, category, image, and location
- Store all complaint and user data locally in the browser using localStorage
- Provide a personal dashboard for citizens to view and track their filed complaints
- Provide an authority/admin view to manage, prioritize, and update complaint statuses
- Implement CSV export functionality to download all complaint data as a portable file
- Implement CSV import functionality to restore complaint data from a previously exported file
- Build a fully responsive, mobile-first interface optimized for smartphone users
- Demonstrate the complete complaint lifecycle: Pending → In Review → In Progress → Resolved

Scope

This Phase 1 release of Gram-Sewa is scoped as a frontend-only prototype with full UI functionality. The application supports five complaint categories: Road, Water Supply, Electricity, Sanitation, and Other. Users can register and log in (credentials stored in localStorage), file complaints with image attachments (stored as Base64 strings), and track resolution status. An admin view allows status updates and priority assignment. Data persistence is limited to the current browser instance via localStorage. To mitigate this limitation, the application provides a CSV export/import feature that allows users to save their data externally and reload it in any browser or device — ensuring no data loss across sessions.

Future phases will introduce a Node.js/Express.js backend with MongoDB for centralized multi-user data storage, real-time notifications, and map-based visualizations.

System Requirements

Hardware Requirements:

- Computer or smartphone with minimum 2 GB RAM
- Any modern processor (Intel/AMD/ARM)
- Storage: Minimal — application runs in browser
- Internet connection (only for initial page load if hosted; works offline after loading)

Software Requirements:

- Operating System: Windows 10/11, macOS, Linux, Android, or iOS
- Browser: Google Chrome 90+, Mozilla Firefox 90+, Microsoft Edge 90+, or Safari 15+
- Development Tools (for building):
 - Node.js v18+ with npm (for development and build only)
 - React 18+ (UI library)
 - Tailwind CSS 3+ (utility-first CSS framework)
 - Vite (build tool and dev server)
 - IDE: Visual Studio Code (recommended)
 - Version Control: Git and GitHub

Methodology

Gram-Sewa follows an iterative development approach. Since the application is entirely frontend-based, all data operations are performed through JavaScript interacting with the browser's localStorage API. The workflow is structured as follows:

Step 1 — User Registration and Login: Users register by providing their name, phone number, role (citizen or authority), and a password. The credentials are serialized as JSON and stored in localStorage under a gramsewa_users key. On login, the entered credentials are validated against stored records. The logged-in user's session is maintained in localStorage until they explicitly log out.

Step 2 — Filing a Complaint: The citizen navigates to the "Report Issue" page and fills out a form: selects a category (Road / Water / Electricity / Sanitation / Other), enters a title and description, uploads a photograph (converted to a Base64 data URL via the FileReader API), and enters the village/ward name. On submission, the complaint object is assigned a unique ID (timestamp-based), given a default status of "Pending," and appended to the gramsewa_complaints array in localStorage.

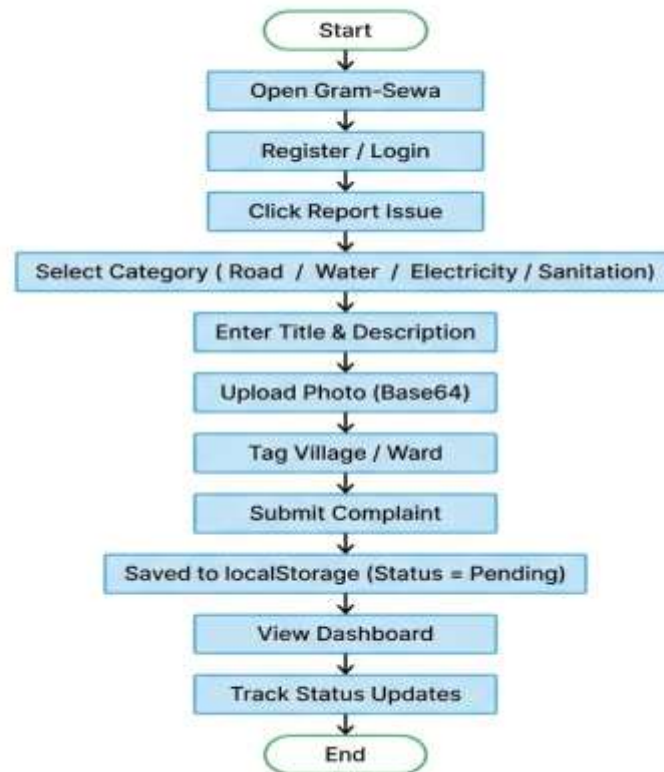
Step 3 — Viewing and Tracking Complaints: The citizen dashboard fetches all complaints from localStorage filtered by the logged-in user's ID. Each complaint is rendered as a card showing title, category, status badge (color-coded), uploaded image thumbnail, and filing date. Citizens can see real-time status changes made by the authority.

Step 4 — Authority Actions: The authority logs in and accesses the admin dashboard, which displays all complaints from localStorage. The authority can filter by category, status, or village. They can open a complaint, assign a priority (Low / Medium / High / Critical), update its status (In Review / In Progress / Resolved), and add resolution remarks. Changes are written back to localStorage immediately.

Step 5 — CSV Export and Import: The export function reads all complaints from localStorage, converts them to CSV format (with columns: ID, Title, Description, Category, Status, Priority, Village, Ward, Filed By, Date, Remarks), and triggers a browser download. The import function accepts a CSV file, parses it using JavaScript, and writes the records into localStorage — merging with or replacing existing data.

Flowchart

Citizen Flow



Authority Flow



Component Structure

Gram-Sewa uses a component-based architecture with React. All state management is handled via React hooks (useState, useEffect, useContext) and localStorage serves as the persistence layer.

Core Components:

App.jsx

- State: currentUser (from localStorage), currentRoute
- Methods: Wraps app in AuthContext provider, manages routing via React Router

AuthContext.jsx

- State: user, isAuthenticated, role
- Methods: login() — validates against localStorage users, register() — adds to localStorage users, logout() — clears session

LoginPage.jsx / RegisterPage.jsx

- State: formData (name, phone, password, role)
- Methods: handleSubmit() — reads/writes users array from/to localStorage

ComplaintForm.jsx

- State: title, description, category, imageBase64, village, ward
- Methods: handleImageUpload() — uses FileReader to convert to Base64, handleSubmit() — creates complaint object and appends to localStorage

ComplaintCard.jsx

- Props: complaint (title, status, category, imageBase64, createdAt)
- Methods: Renders styled card with color-coded status badge, category icon, and Base64 image thumbnail

CitizenDashboard.jsx

- State: myComplaints[] (filtered from localStorage by userId), filterStatus
- Methods: loadComplaints() — reads from localStorage and filters by current user

AdminDashboard.jsx

- State: allComplaints[], stats (total, pending, resolved), filters
- Methods: loadAllComplaints() — reads from localStorage, updateStatus() — modifies complaint in localStorage, assignPriority()

CSVManager.jsx

- Methods: exportToCSV() — reads localStorage, builds CSV string, triggers download via Blob/URL.createObjectURL, importFromCSV() — accepts file input, parses CSV rows, writes to localStorage

StatusBadge.jsx

- Props: status (Pending / In Review / In Progress / Resolved)
- Methods: Returns a Tailwind-styled badge with appropriate color (yellow / blue / orange / green)

Implementation

The entire application is built and bundled using Vite with the React template. Tailwind CSS is configured via `tailwind.config.js` for utility-first styling with a custom color palette reflecting the civic/rural theme.

Authentication Implementation: User registration data is stored in `localStorage` under the key `gramsewa_users` as a JSON array. Each user object contains an ID, name, phone, hashed password (using a lightweight client-side hash for demonstration), and role (citizen or authority). Login validates input against stored records and sets `gramsewa_current_user` in `localStorage` to maintain the session.

Complaint Management Implementation: Complaints are stored under `gramsewa_complaints` as a JSON array. Each complaint object includes a unique ID (generated from `Date.now()`), title, description, category, status, priority, location (village and ward), the filing user's ID, an image stored as a Base64 data URL string, timestamps (`createdAt`, `updatedAt`), and optional resolution remarks. The `ComplaintForm` component uses the `FileReader` API to read the uploaded image and convert it to a Base64 string before storing. The `CitizenDashboard` component filters this array by user ID, while the `AdminDashboard` loads the full array.

CSV Export/Import Implementation: The export function reads the `gramsewa_complaints` array from `localStorage`, constructs a CSV string with proper header rows and comma-escaped values, creates a Blob object with MIME type `text/csv`, generates a temporary download URL via `URL.createObjectURL()`, and programmatically clicks a hidden anchor tag to trigger the download. The import function uses an HTML file input element, reads the selected CSV file via `FileReader`, splits content by newlines and commas (handling quoted fields), maps each row to a complaint object, and writes the array to `localStorage`. A confirmation prompt asks the user whether to merge with or replace existing data.

Sample Input/Output:

Citizen Files a Complaint: Input: Category = "Road", Title = "Broken road near primary school", Description = "Large pothole causing accidents", Image = `pothole.jpg` (uploaded and converted to Base64), Village = "Sundarpur", Ward = "Ward 3". Output: Complaint saved to `localStorage` with ID #1713580000000, Status = "Pending". The citizen's dashboard displays a new card with a yellow "Pending" badge.

Authority Updates Status: Input: Opens complaint #1713580000000, sets Priority = "High", Status = "In Progress", Remarks = "Repair team notified". Output: `localStorage` record updated. Citizen dashboard now shows a blue "In Progress" badge with the authority's remarks.

CSV Export: Input: User clicks "Export Data" button. Output: Browser downloads `gramsewa_complaints_2026-04-20.csv` containing all complaint records.

Future Enhancements

The following enhancements are planned for future iterations of Gram-Sewa:

- **Backend Integration:** Add Express.js REST API with MongoDB for centralized, multi-user data storage (Phase 2)
- **Real Authentication:** Replace `localStorage`-based login with JWT tokens and `bcrypt` password hashing
- **Map Integration:** Display complaint locations on an interactive map using `Leaflet.js`
- **Push Notifications:** SMS/email alerts when complaint status changes via `Twilio` or `Nodemailer`
- **Upvoting System:** Allow community members to upvote complaints to signal urgency
- **Progressive Web App (PWA):** Service worker for offline-first functionality and home screen installation
- **Multilingual Support:** Hindi and regional language interfaces for inclusive access
- **AI Categorization:** Auto-classify complaint categories from uploaded images using a machine learning API
- **Government Portal Integration:** Connect with CPGRAMS/Samadhaan for escalation to higher authorities

Gantt Chart

Planning & Structure

The development of Gram-Sewa Phase 1 (frontend-only) is organized into a 6-week timeline:

Week 1: Requirement Analysis & UI Design Define user stories, create wireframes for citizen and admin interfaces, design the complaint form layout, finalize the localStorage data schema, and set up the project repository with React + Vite + Tailwind CSS.

Week 2: Core UI Components Build all page components — LoginPage, RegisterPage, HomePage, ComplaintForm, CitizenDashboard, AdminDashboard. Implement React Router for navigation and create reusable components (StatusBadge, ComplaintCard, Navbar).

Week 3: localStorage Integration Implement the data persistence layer — user registration/login with localStorage, complaint CRUD operations, image-to-Base64 conversion and storage, session management, and status update logic.

Week 4: CSV Export/Import & Admin Features Build the CSVManager component with export and import functionality. Implement admin filtering, priority assignment, status updates, and analytics summary (total/pending/resolved counts).

Week 5: Testing & Responsiveness Test all user flows end-to-end, fix mobile responsiveness issues with Tailwind breakpoints, validate localStorage limits with multiple complaints, test CSV round-trip (export → import), and cross-browser compatibility.

Week 6: Documentation & Deployment Write project documentation (synopsis report), deploy on Vercel/Netlify/GitHub Pages as a static site, record demo video, and prepare for project presentation.

Time Management

Each development task has a defined start and end boundary within the 6-week timeline. UI development (Week 2) and localStorage integration (Week 3) are sequential since data logic depends on component structure. The CSV module (Week 4) runs in parallel with admin dashboard refinements. Week 5 provides dedicated buffer time for bug fixes and edge-case handling. The final week is reserved purely for documentation and deployment to prevent last-minute rushes.

Conclusion

Gram-Sewa successfully demonstrates the development of a full-stack web application that addresses a real-world civic problem in rural India. The project implements a complete complaint lifecycle management system — from citizen-driven issue reporting with photographic evidence to authority-managed resolution tracking with priority assignment and analytics.

Built entirely with React and Tailwind CSS, the application delivers a fully functional complaint lifecycle — from user registration and issue filing with photo evidence to authority-managed resolution tracking — all persisting data locally in the browser via localStorage.

The CSV export/import feature addresses the primary limitation of client-side storage by enabling data portability across browsers and devices. The responsive, mobile-first design ensures accessibility for rural smartphone users — the platform's primary audience.

This Phase 1 implementation validates the user experience, interface design, and core workflows of Gram-Sewa. It establishes a solid frontend foundation that can be seamlessly extended with a Node.js/Express.js backend and MongoDB database in Phase 2, enabling multi-user support, centralized data storage, and real-world deployment at the Gram Panchayat level.

References

- [1] A. Banks and E. Porcello, *Learning React: Modern Patterns for Developing React Apps*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2022.
- [2] S. Grider, *Modern React with Redux*, 1st ed. San Francisco, CA: Udemy Publications, 2023.
- [3] Meta Platforms, "React Documentation," React. [Online]. Available: <https://react.dev>
- [4] Tailwind Labs, "Tailwind CSS Documentation," Tailwind CSS. [Online]. Available: <https://tailwindcss.com/docs>
- [5] Evan You, "Vite Documentation," Vite. [Online]. Available: <https://vitejs.dev/guide>
- [6] Mozilla Developer Network, "Web Storage API — localStorage," MDN. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- [7] Mozilla Developer Network, "FileReader API," MDN. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/FileReader>

Journal Article:

- [8] P. K. Singh and A. Sharma, "Role of ICT in Rural Governance and Development in India," *International Journal of Computer Applications*, vol. 120, no. 18, pp. 32-37, 2015.